GPU – ATOMICS AND HISTOGRAM

2026 SPRING

JINHO LEE

ECE, SNU

Accelerated Intelligent Systems Lab.

IN PREVIOUS EXAMPLES..

- We did not use any locks
- Every operation was local to each threadblock
  - ‘Privatization’
  - This is good, but not always possible
- Some serialization is needed in many cases
- Atomics is usually a better option than locks

RECALL ATOMICS IN C++

```
#include <iostream>
#include <thread>
#include <vector>
#include <atomic>
#include <assert.h>

std::atomic<int> output

void worker(int* input, int start, int size)
{
  for(int i=0;i<size;i++) {
    if(input[start+i]==0) {
      output+=1;
    }
  }
}
```

- If the job between lock-unlock is very simple, you can use std::atomic
- There are special hardware instructions for this
- And it's much faster!
  - Warning: still *much* slower than normal operations

ATOMIC OPERATIONS IN CUDA

- Function calls that are translated into single instructions (a.k.a. *intrinsics*)
  - Atomic add, sub, inc, dec, min, max, exch (exchange), CAS (compare and swap)
  - Read CUDA C programming Guide for details

- Atomic Add

  int atomicAdd(int **address**, int **val**);*

  reads the 32-bit word **old** pointed to by **address** in global or shared memory, computes **(old + val)**, and stores the result back to memory at the same address. The function returns **old**.

MORE ATOMIC ADDS IN CUDA

- Unsigned 32-bit integer atomic add

  unsigned int atomicAdd(unsigned int address, unsigned int val);*

- Unsigned 64-bit integer atomic add

  unsigned long long int atomicAdd(unsigned long long int address, unsigned long long int val);*

- Single-precision floating-point atomic add
  - float atomicAdd(float* address, float val);

HISTOGRAM

- Counting how many elements fall into certain bins
- Example: "**Programming Massively Parallel**", each bin is an alphabet

HISTOGRAM

- Counting how many elements fall into certain bins
- Example: "**Programming Massively Parallel**", each bin is an alphabet

4																					
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V

HISTOGRAM

- Counting how many elements fall into certain bins
- Example: "**Programming Massively Parallel**", each bin is an alphabet

4				1																		
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V

HISTOGRAM

- Counting how many elements fall into certain bins
- Example: "**Programming Massively Parallel**", each bin is an alphabet

4				1		1															
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V

HISTOGRAM

- Counting how many elements fall into certain bins
- Example: "**Programming Massively Parallel**", each bin is an alphabet

4				1		1		2			4	3	1	1	2		3	2			
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V

HISTOGRAM

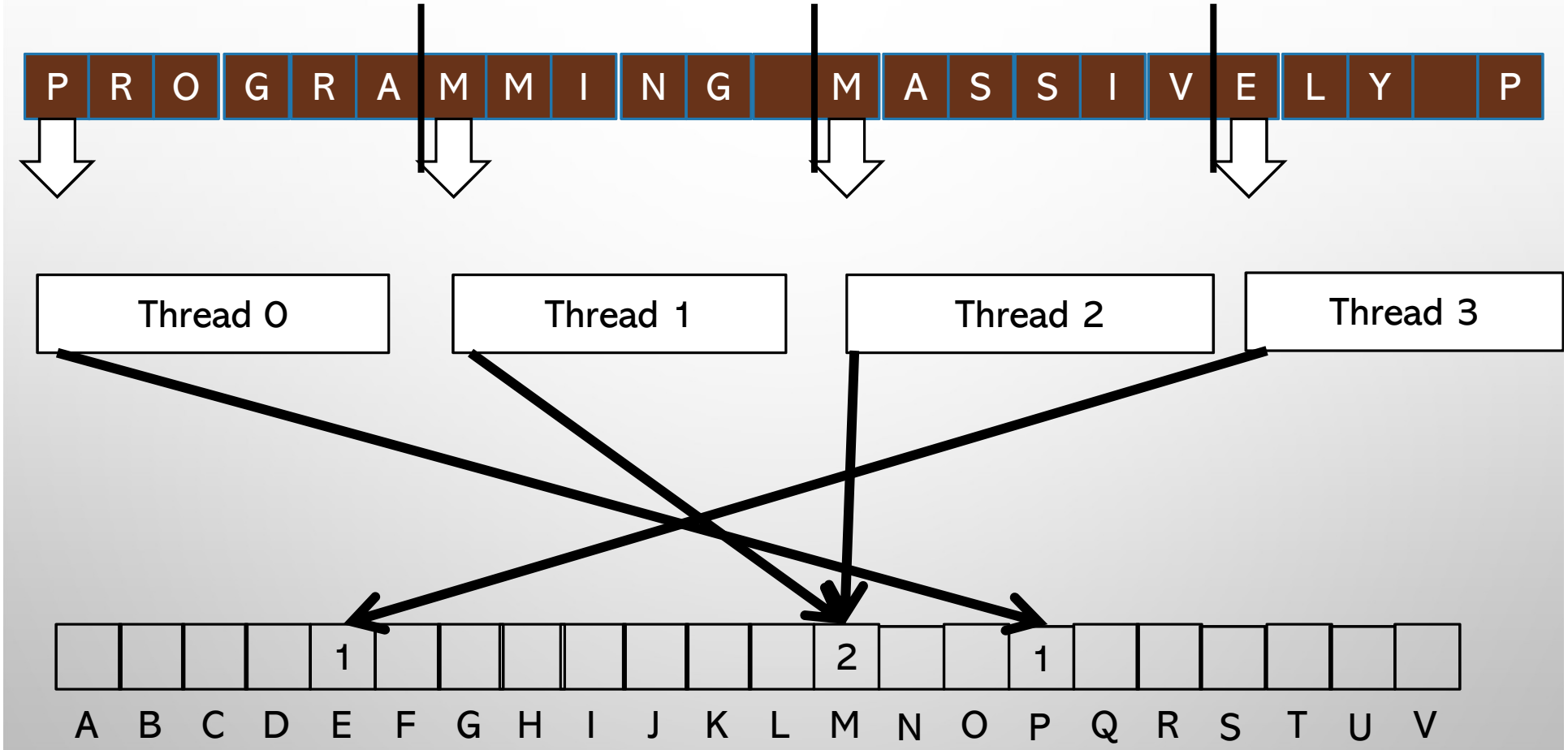
- Counting how many elements fall into certain bins
- Example: "**Programming Massively Parallel**", each bin is an alphabet

- **Let's parallelize histogramming a long sentence**
  - **Using atomic adds**

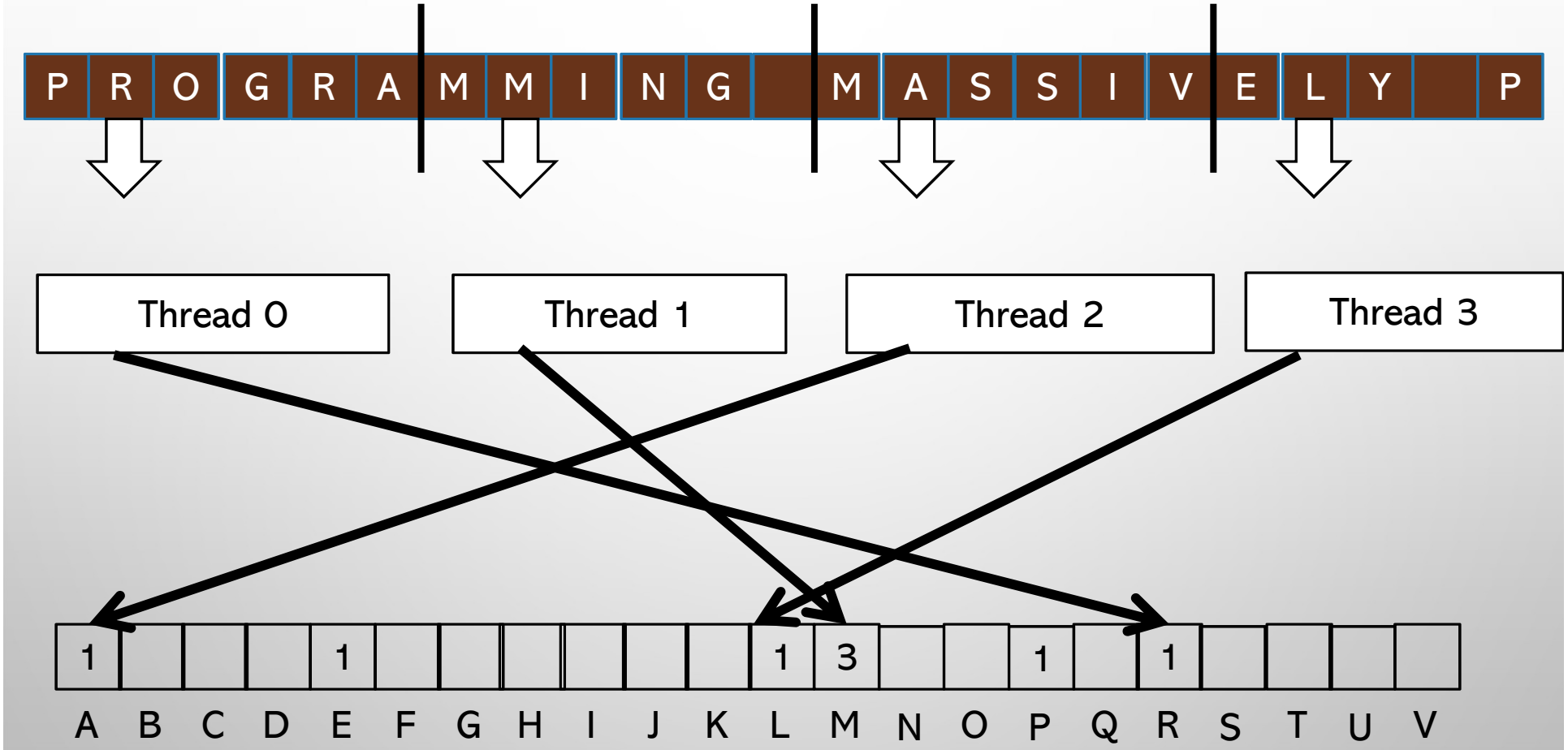
- **We will start from a very stupid kernel**

4				1		1		2			4	3	1	1	2		3	2				
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	

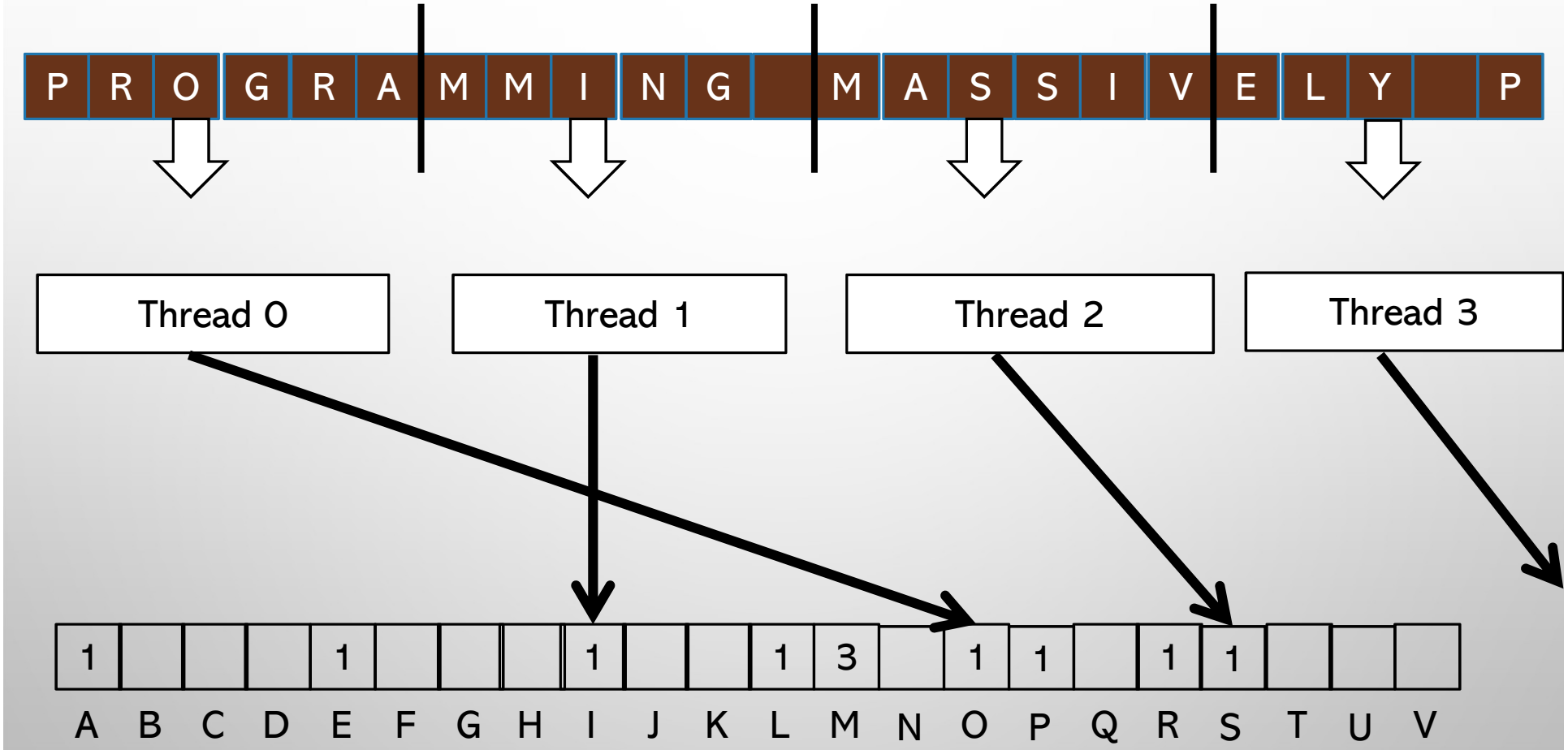
ITERATION #1 – 1ST LETTER IN EACH SECTION



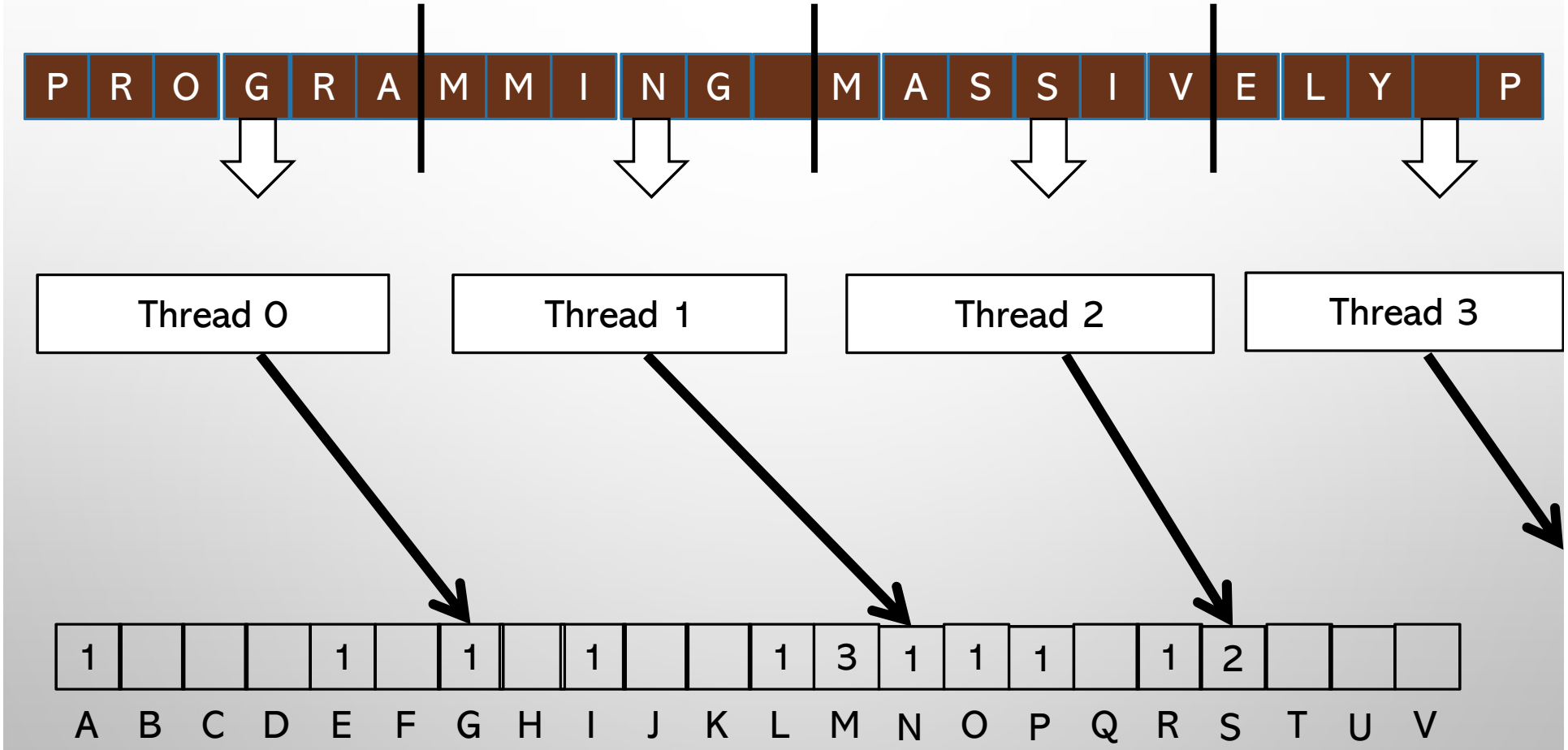
ITERATION #2 – 2ND LETTER IN EACH SECTION



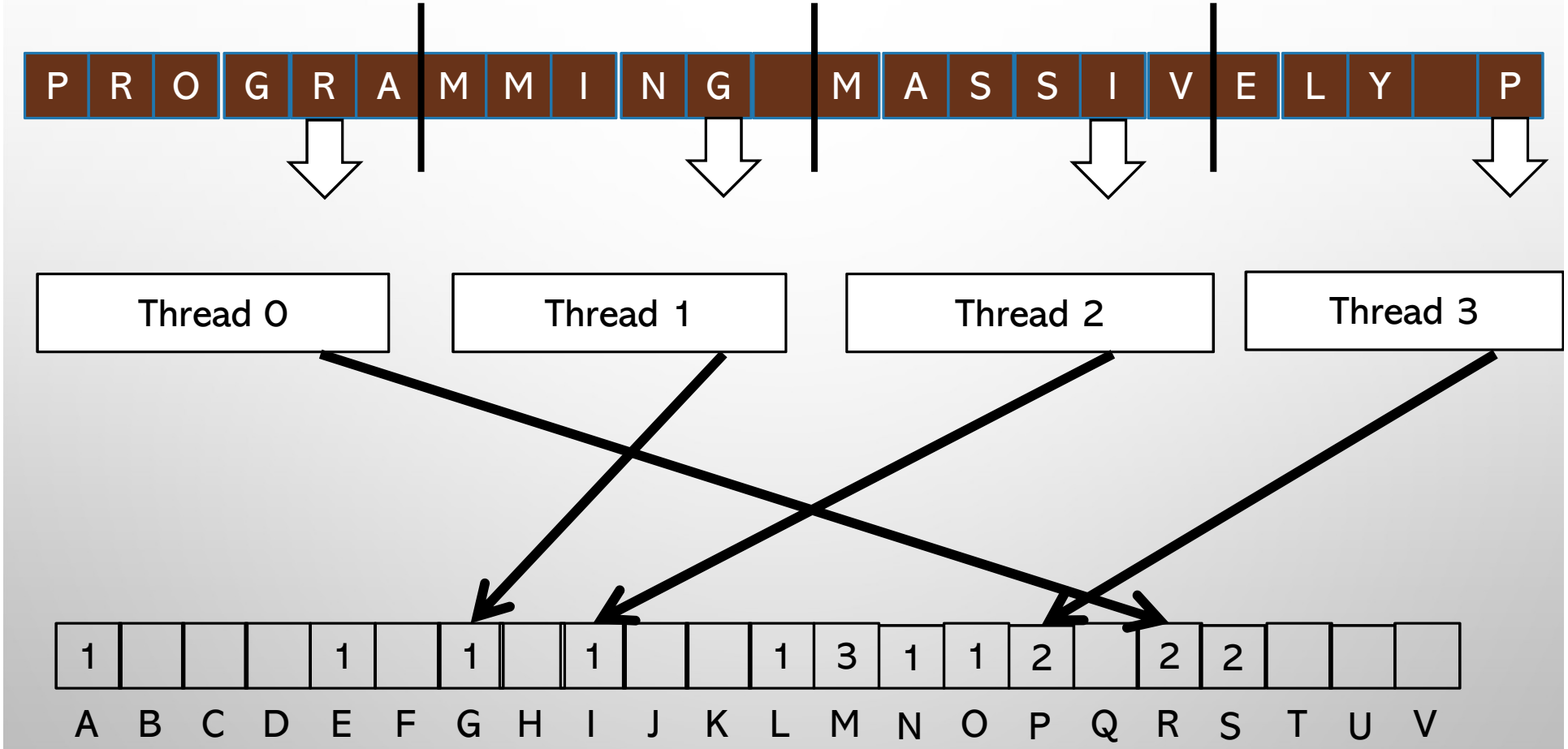
ITERATION #3


P R O G R A M M I N G M A S S I V E L Y P
Thread 0
Thread 1
Thread 2
Thread 3
1 1 1 1 3 1 1 1 1
A B C D E F G H I J K L M N O P Q R S T U V

ITERATION #4


P R O G R A M M I N G M A S S I V E L Y P
Thread 0
Thread 1
Thread 2
Thread 3
1 1 1 1 1 3 1 1 1 1 2
A B C D E F G H I J K L M N O P Q R S T U V

ITERATION #5


P R O G R A M M I N G M A S S I V E L Y P
Thread 0
Thread 1
Thread 2
Thread 3
1 1 1 1 1 3 1 1 2 2 2
A B C D E F G H I J K L M N O P Q R S T U V

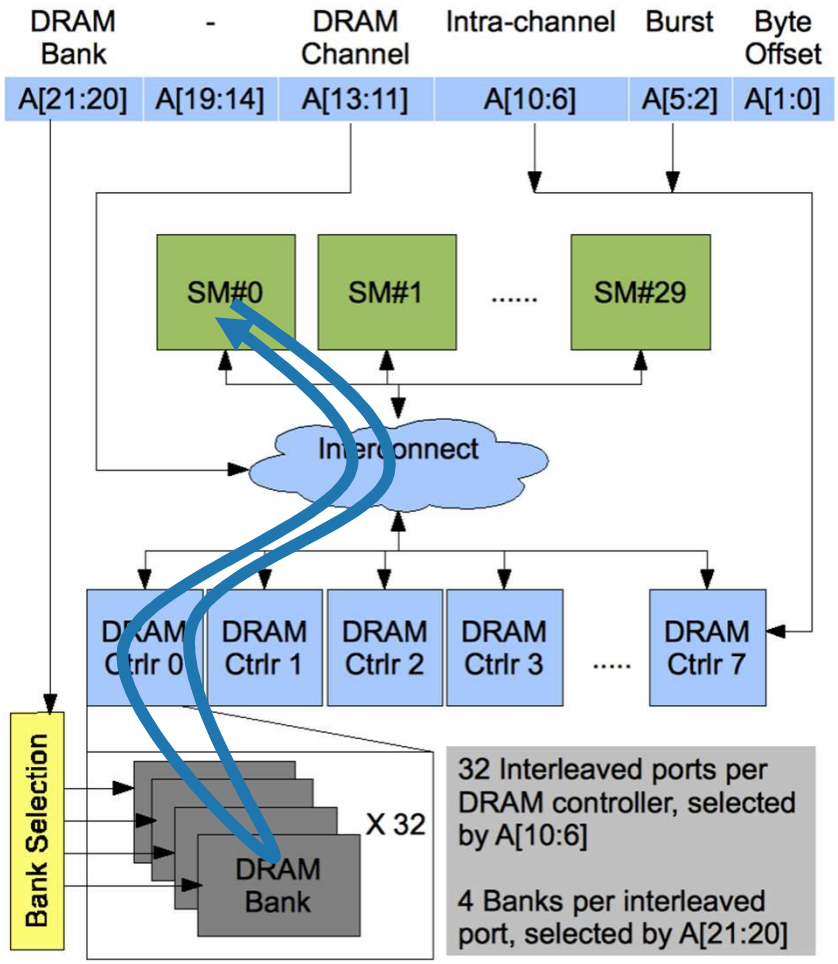
FIRST TRY

```
__global__ void histogram_v0(int *input, int *result, int N) {
  // Calculate global thread ID
  int tid = blockIdx.x * blockDim.x + threadIdx.x;
  // Calculate the amount of work per thread
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;

  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    // Calculate the position in the alphabet
    int input_index = tid*chunksize + i;
    bin_position = input[input_index]; //assume the items are [0 ~ BIN-1]
    atomicAdd(&result[bin_position], 1);
  }
}
```

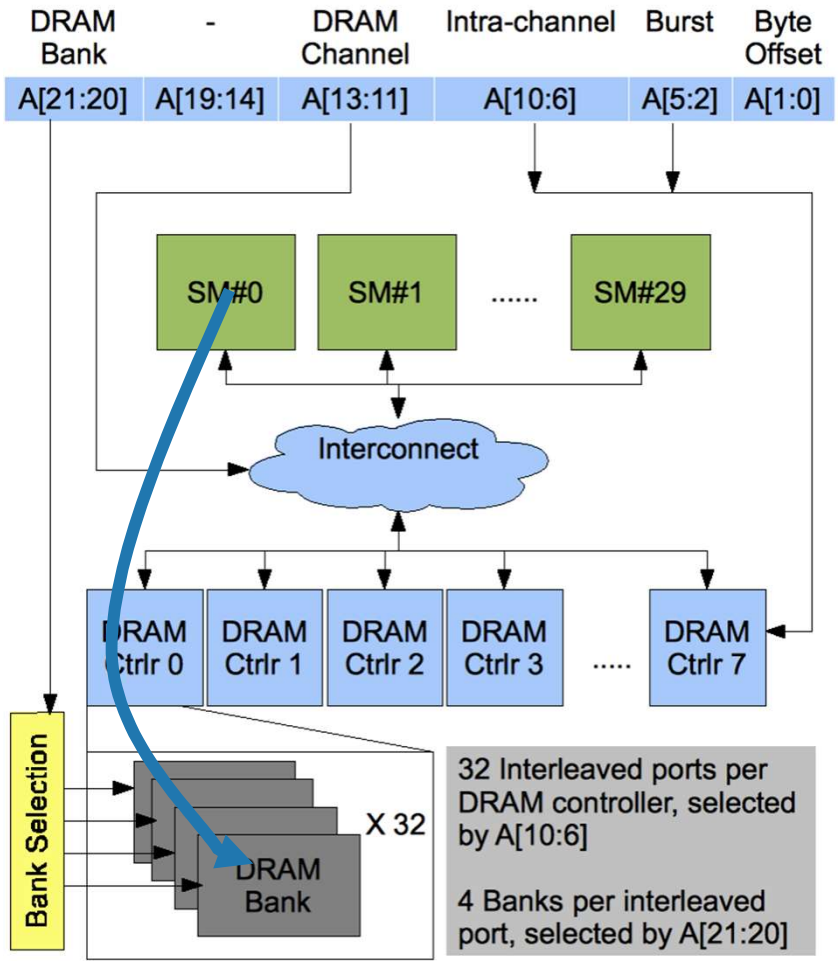
- You need some boundary handling when N is not power of 2
- What can you improve?

ATOMIC OPERATIONS ON DRAM



- An atomic operation starts with a read, with a latency of a few hundred cycles

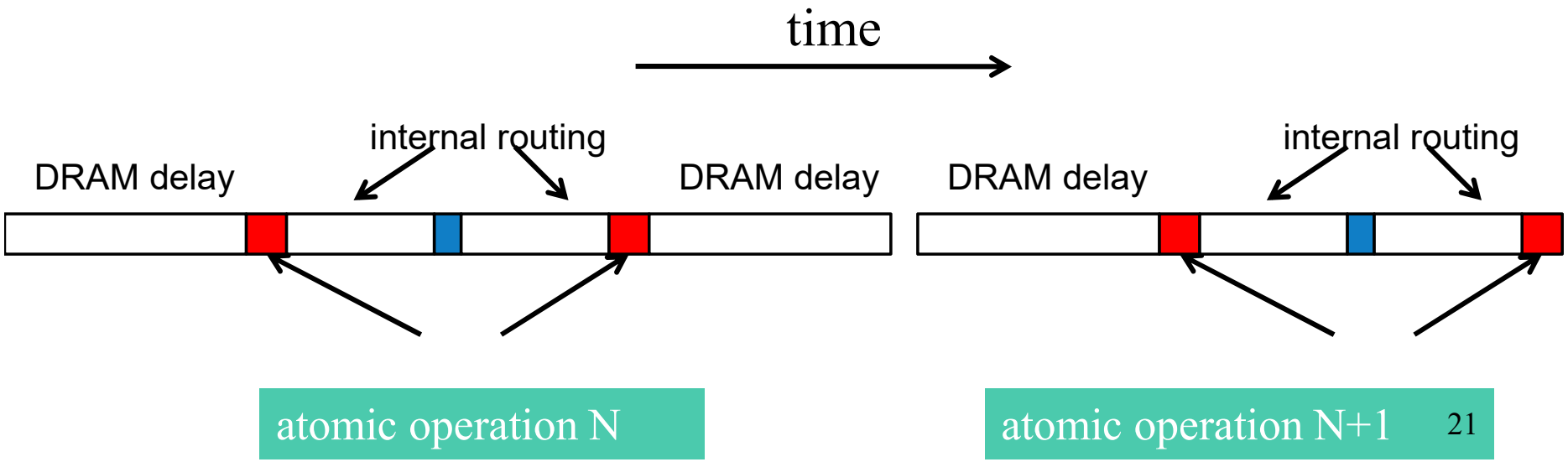
ATOMIC OPERATIONS ON DRAM



- An atomic operation starts with a read, with a latency of a few hundred cycles
- The atomic operation ends with a write, with a latency of a few hundred cycles
- During this whole time, no one else can access the location

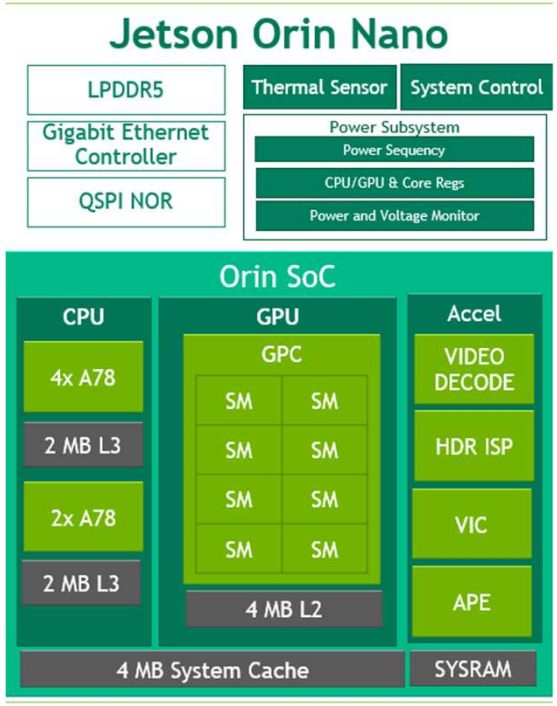
ATOMIC OPERATIONS ON DRAM

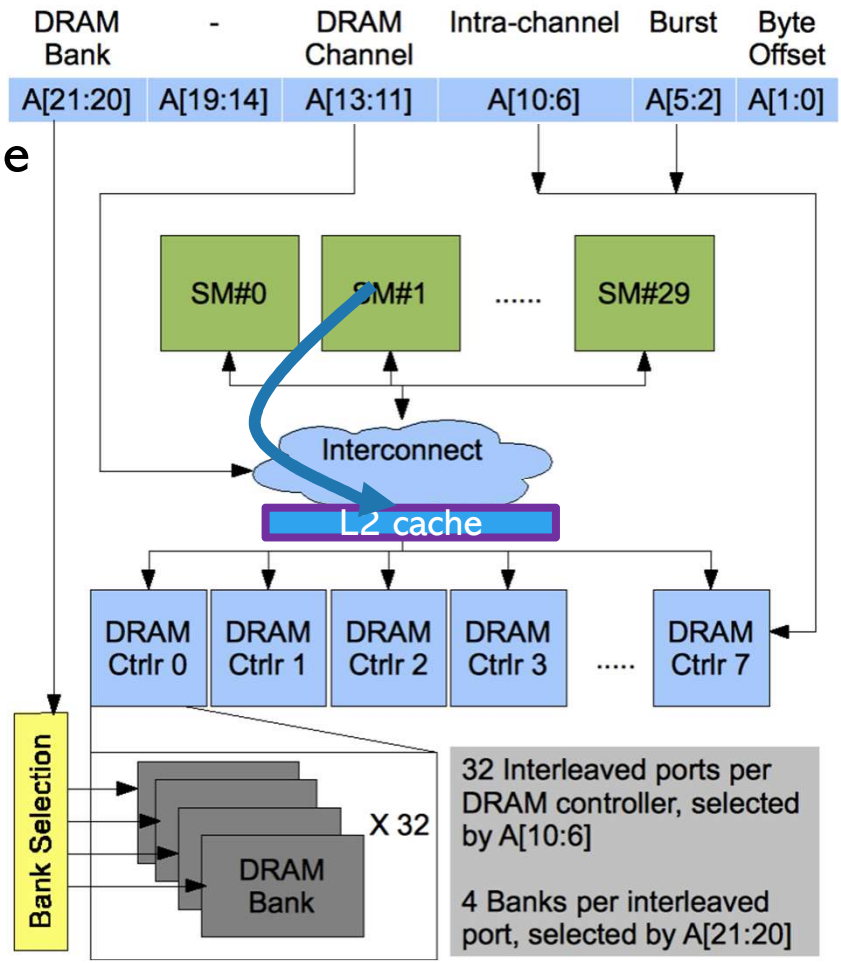
- Each Load-Modify-Store has two full memory access delays
  - All atomic operations on the same variable (RAM location) are serialized



IN FERMI (2010)

- Atomics on L2 cache has been implemented
- You get speedup for free with the same code if the element is found on the L2 cache
  - But the L2 cache is small…..

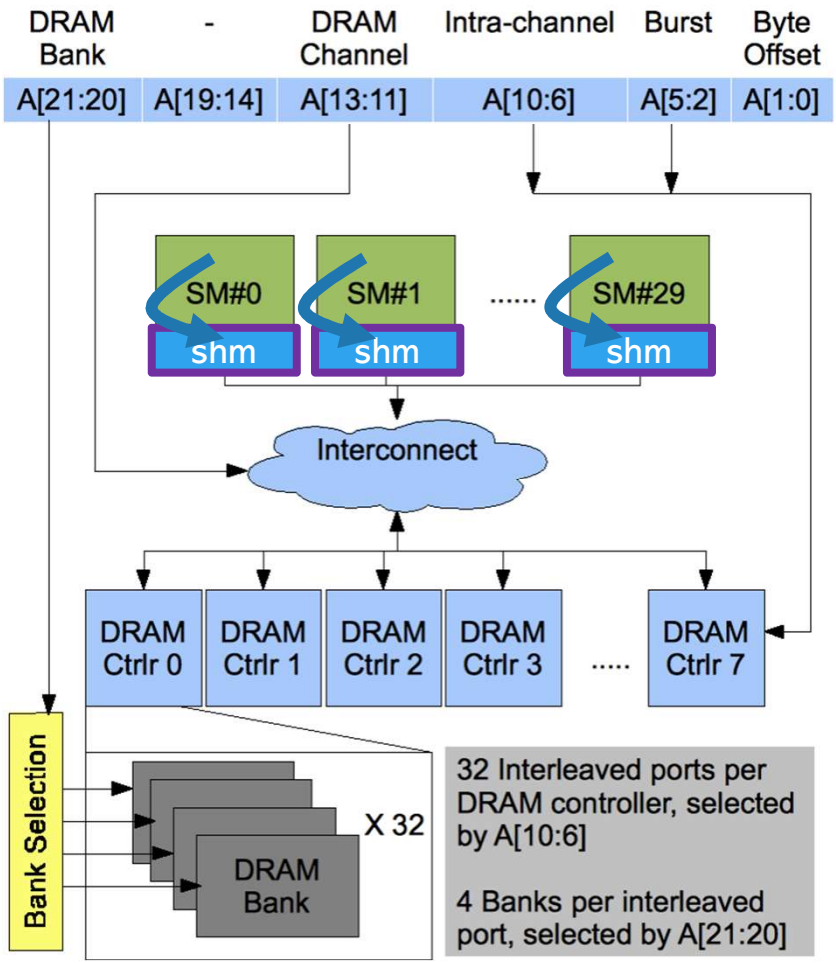




IN MAXWELL (2014)

- Atomics on shared memory has been implemented
- Users can control it now
- Same way, but with much shorter latency

- But the shared memory is private to threadblocks
- You need some more work



SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- Local histogram approach

SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- Implement local bins in shared memory

SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- Initialize shared memory

SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- #BINS might be larger than #threads
- If #BINS is small, (local_tid < BINS) is enough

SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- Make sure everyone is done

SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- Find the bin position

SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- Atomic_add is done on the shared memory

SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- You need to syncthreads() only after the loop is finished, not every iteration.
- Why?

SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- Now, dump local histogram into the global histogram

SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- the two atomicAdd() invocations are the same, but have different performance
  - One is on shared memory, the other is on global memory

SECOND TRY

```
__global__ void histogram_v1 (int *input, int *result, int N) {
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int chunksize = (N-1) / (gridDim.x * blockDim.x) + 1;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();

  // Calculate the local histogram in smem
  int bin_position;
  for (int i = 0; i < chunksize; i++) {
    unsigned int input_index = global_tid*chunksize + i;
    bin_position = input[input_index];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();

  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- Notice the difference in work distribution
- Instead of elements, bins are assigned to threads, similar to the initiliazation
- Again, if #BINS is small, (local_tid < BINS) is enough

PERFORMANCE

- with threadblock size tuning

For 10 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	1.028	2.044	4.078	8.141	16.266	32.533
v1	0.056	0.126	0.189	0.366	0.715	1.417

22x

For 1000 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	0.284	0.557	1.107	2.199	4.376	8.737
v1	0.041	0.072	0.126	0.238	0.461	0.901

- Much more conflicts with 10 bins (large speedup on v1)
- Less conflicts with 1000 bins (smaller speedup on v1.
  v0 has higher speedup from 10 bins → 1000 bins)

PERFORMANCE

- with threadblock size tuning

For 10 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	1.028	2.044	4.078	8.141	16.266	32.533
v1	0.056	0.126	0.189	0.366	0.715	1.417

For 1000 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	0.284	0.557	1.107	2.199	4.376	8.737
v1	0.041	0.072	0.126	0.238	0.461	0.901

9x

- Much more conflicts with 10 bins (large speedup on v1)
- Less conflicts with 1000 bins (smaller speedup on v1.
  v0 has higher speedup from 10 bins → 1000 bins)

PERFORMANCE

- with threadblock size tuning

For 10 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	1.028	2.044	4.078	8.141	16.266	32.533
v1	0.056	0.126	0.189	0.366	0.715	1.417

3.7x

For 1000 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	0.284	0.557	1.107	2.199	4.376	8.737
v1	0.041	0.072	0.126	0.238	0.461	0.901

- Much more conflicts with 10 bins (large speedup on v1)
- Less conflicts with 1000 bins (smaller speedup on v1.
  v0 has higher speedup from 10 bins → 1000 bins)

PERFORMANCE

- with threadblock size tuning

For 10 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	1.028	2.044	4.078	8.141	16.266	32.533
v1	0.056	0.126	0.189	0.366	0.715	1.417

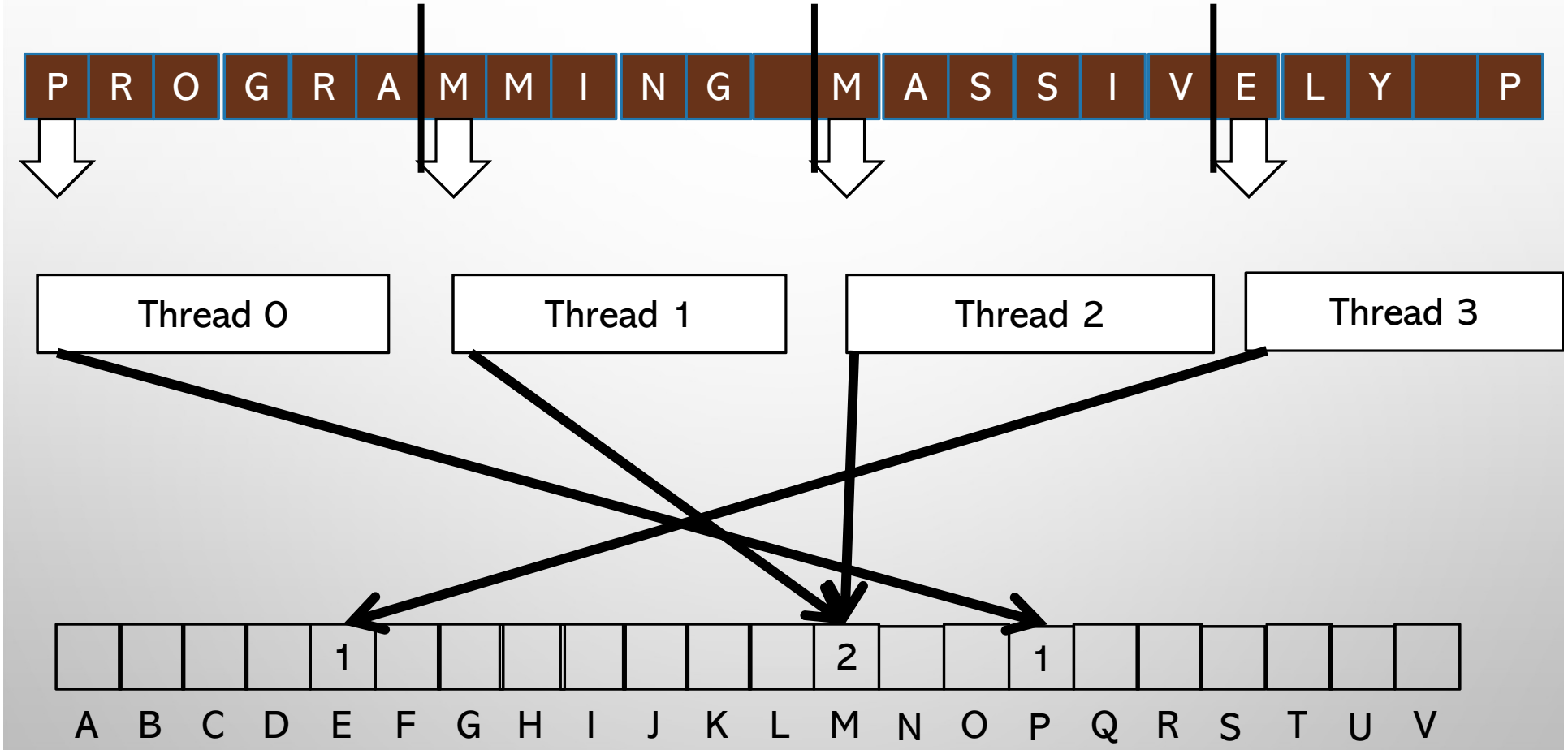
1.5x

For 1000 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	0.284	0.557	1.107	2.199	4.376	8.737
v1	0.041	0.072	0.126	0.238	0.461	0.901

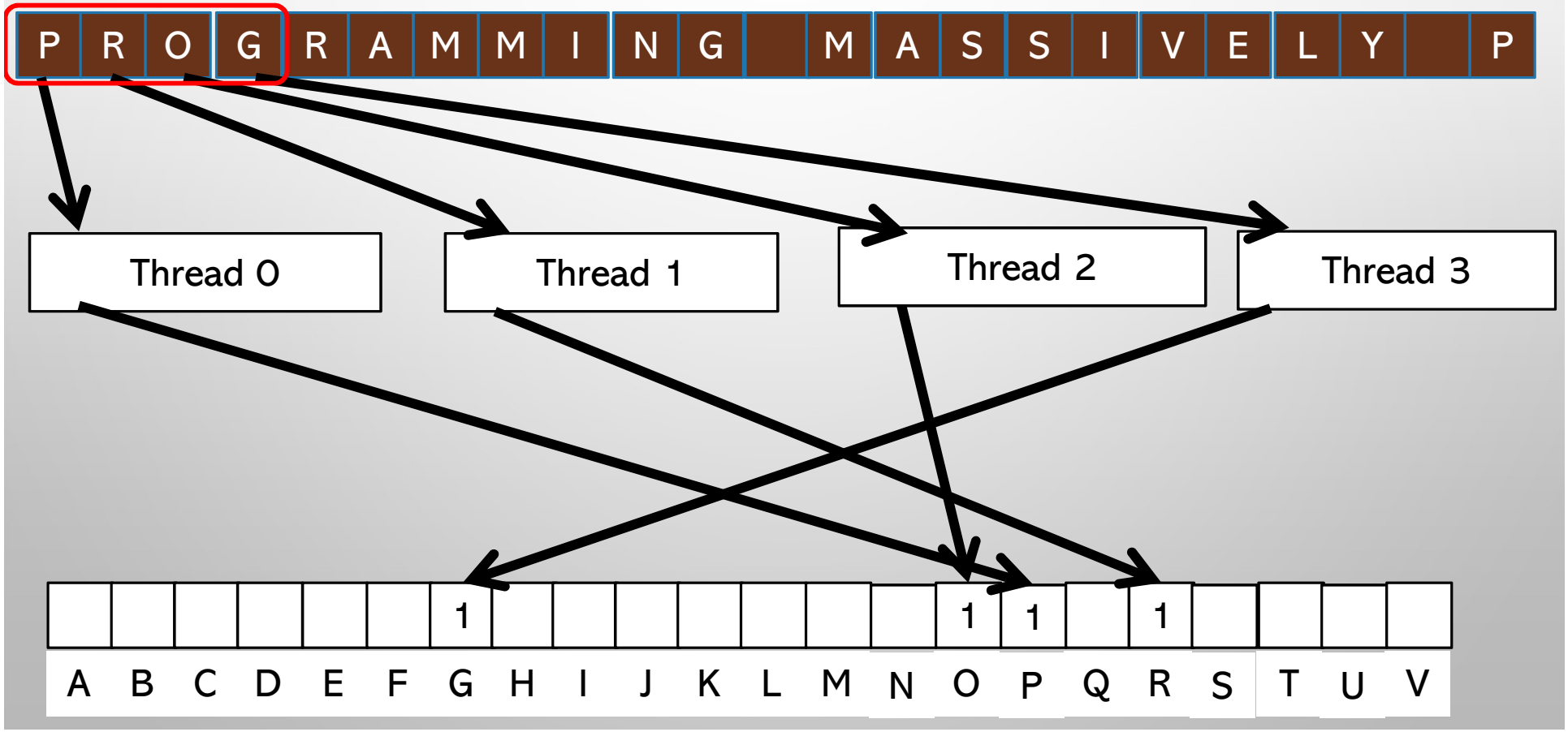
- Much more conflicts with 10 bins (large speedup on v1)
- Less conflicts with 1000 bins (smaller speedup on v1.
  v0 has higher speedup from 10 bins → 1000 bins)

SOMETHING TRIVIAL: THE READS ARE NOT COALESCED

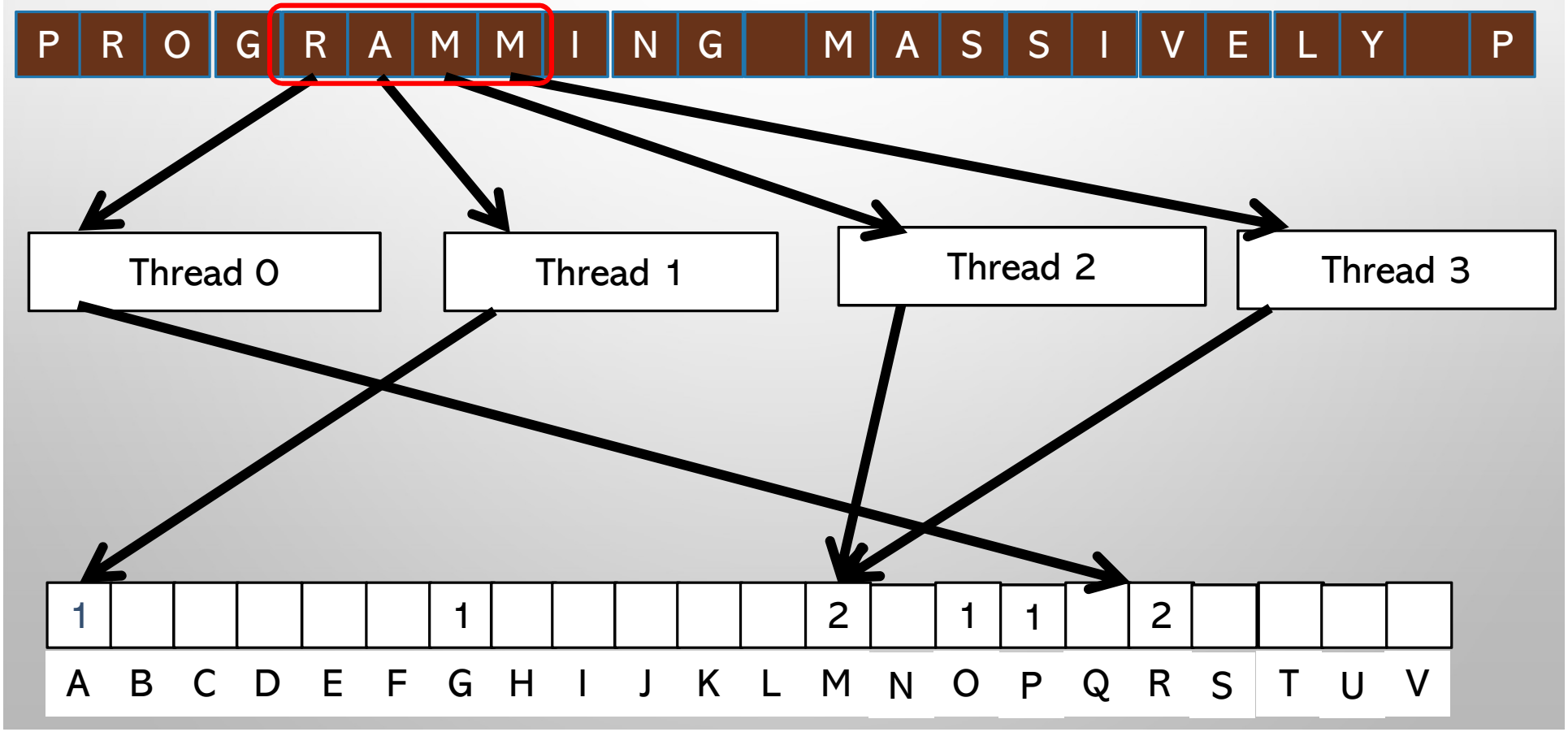


WITH COALESCED READS

- Reads from the input array are not coalesced
  - Assign inputs to each thread in a strided pattern
  - Adjacent threads process adjacent input letters



WITH COALESCED READS



THIRD TRY

```
__global__ void histogram_v2 (int *input, int *result, int N) {
  // Calculate thread ID
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();
  // Calculate the local histogram in smem
  int bin_position;
  for (unsigned int i = global_tid; i < N;
                      i += (gridDim.x * blockDim.x)) {
    // Calculate the position
    bin_position = input[i];
    atomicAdd(&s_localbin[bin_position], 1);
  }
  __syncthreads();
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- Just change the indexing

PERFORMANCE

- Some extra speedup

For 10 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	1.028	2.044	4.078	8.141	16.266	32.533
v1	0.056	0.126	0.189	0.366	0.715	1.417
v2	0.047	0.086	0.162	0.309	0.600	1.170

For 1000 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	0.284	0.557	1.107	2.199	4.376	8.737
v1	0.041	0.072	0.126	0.238	0.461	0.901
v2	0.036	0.062	0.113	0.215	0.418	0.815

PERFORMANCE

- V0.5: gmem atomics + coalescing does not help much
- Because global memory atomic operation itself is the bottleneck

For 10 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	1.028	2.044	4.078	8.141	16.266	32.533
V0.5	**1.027**	**2.045**	**4.075**	**8.137**	**16.262**	**32.499**
v1	0.056	0.126	0.189	0.366	0.715	1.417
v2	0.047	0.086	0.162	0.309	0.600	1.170

For 1000 bins,

Version / **N**	2^21	2^22	2^23	2^24	2^25	2^26
v0	0.284	0.557	1.107	2.199	4.376	8.737
V0.5	**0.283**	**0.556**	**1.108**	**2.199**	**4.381**	**8.741**
v1	0.041	0.072	0.126	0.238	0.461	0.901
v2	0.036	0.062	0.113	0.215	0.418	0.815

WE STILL HAVE SOME GLOBAL ATOMICS

```
__global__ void histogram_v2 (int *input, int *result, int N) {
  // Calculate thread ID
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();
  // Calculate the local histogram in smem
  int bin_position;
  for (unsigned int i = global_tid; i < N;
                      i += (gridDim.x * blockDim.x)) {
    // Calculate the position
    bin_position = input[i];
    atomicAdd(&s_localbin[bin_position], 1);
  }
  __syncthreads();
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    atomicAdd(&result[bin_id], s_localbin[bin_id]);
  }
}
```

- What if your global memory atomic is very slow?
  - This was the case for old GPUs..
  - Didn't quite check this on Jetson

KERNEL SPLIT

```
__global__ void histogram_v3_local(int *input, int *result, int N) {
  // Calculate thread ID
  int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
  int local_tid = threadIdx.x;
  int block_id = blockIdx.x;
  __shared__ unsigned int s_localbin[BINS];
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    s_localbin[bin_id]=0;
  }
  __syncthreads();
  // Calculate the local histogram in smem
  int bin_position;
  for (unsigned int i = global_tid; i < N; i += (gridDim.x *
     blockDim.x)) {
    // Calculate the position
    bin_position = input[i];
    atomicAdd(&s_localbin[bin_position ], 1);
  }
  __syncthreads();
  for(int bin_id=local_tid; bin_id < BINS; bin_id += blockDim.x) {
    result[block_id * BINS + bin_id] = s_localbin[bin_id];
  }
```

- Instead of atomic addition, write to different locations of gmem

KERNEL SPLIT

```
__global__ void histogram_v3_global(int *input, int* result, int
     num_blocks) {
  // Calculate global thread ID
  int tid = blockIdx.x * blockDim.x + threadIdx.x;
  // number of threadblock better be one
  for(int bin_id=tid; bin_id < BINS; bin_id += blockDim.x) {
    //we don't use reduce here. parallel reduction might increase
     perf when N is large
    unsigned int total=0;
    for(int i=0;i<num_blocks;i++) {
      total += input[bin_id + i*BINS];
    }
    result[bin_id] = total;
  }
}
```

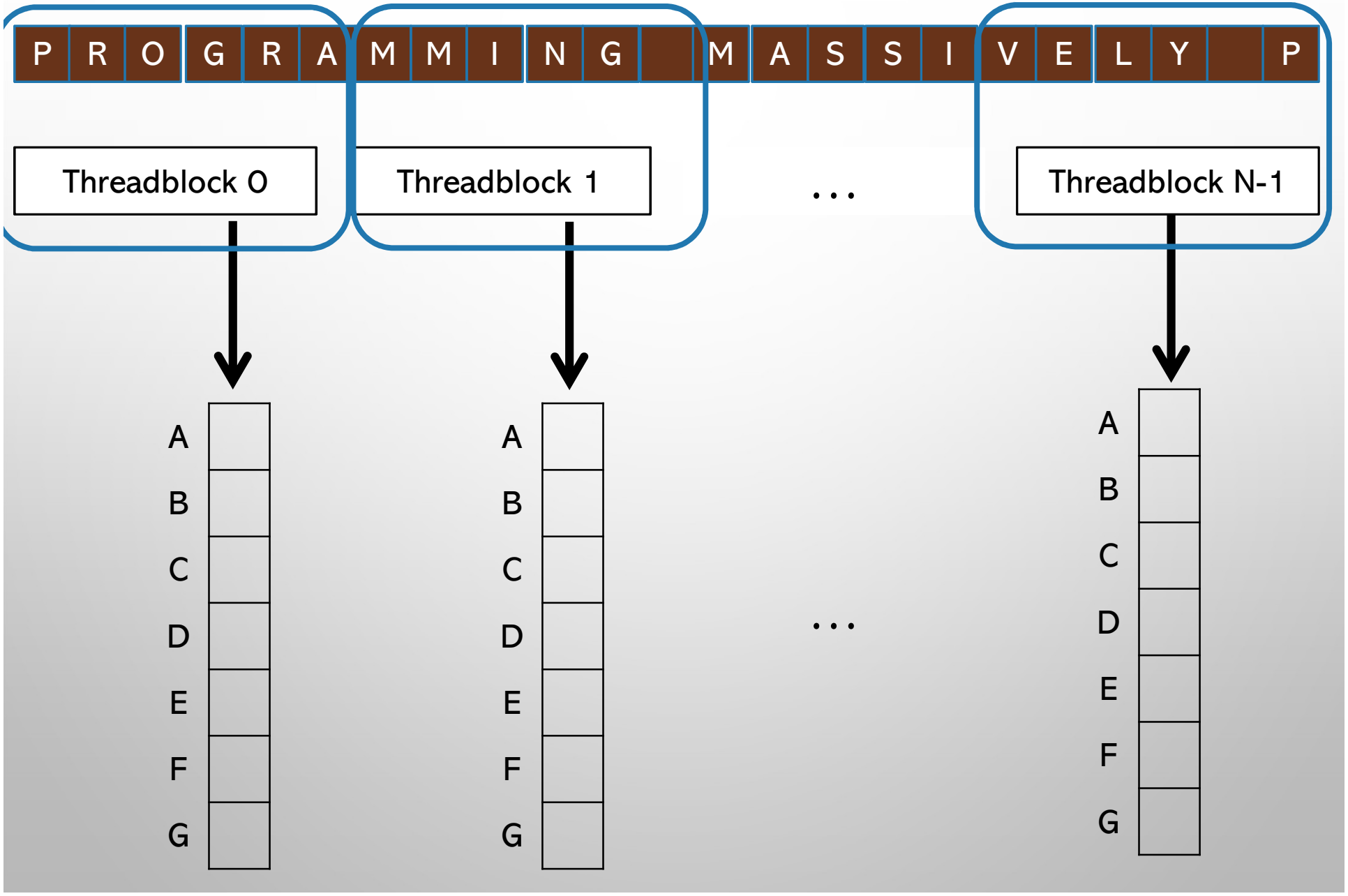
- Next kernel merges the histograms
- A common technique used in many cases
- I didn't try, but parallel reduction might help here depending on the worksize

KERNEL SPLIT

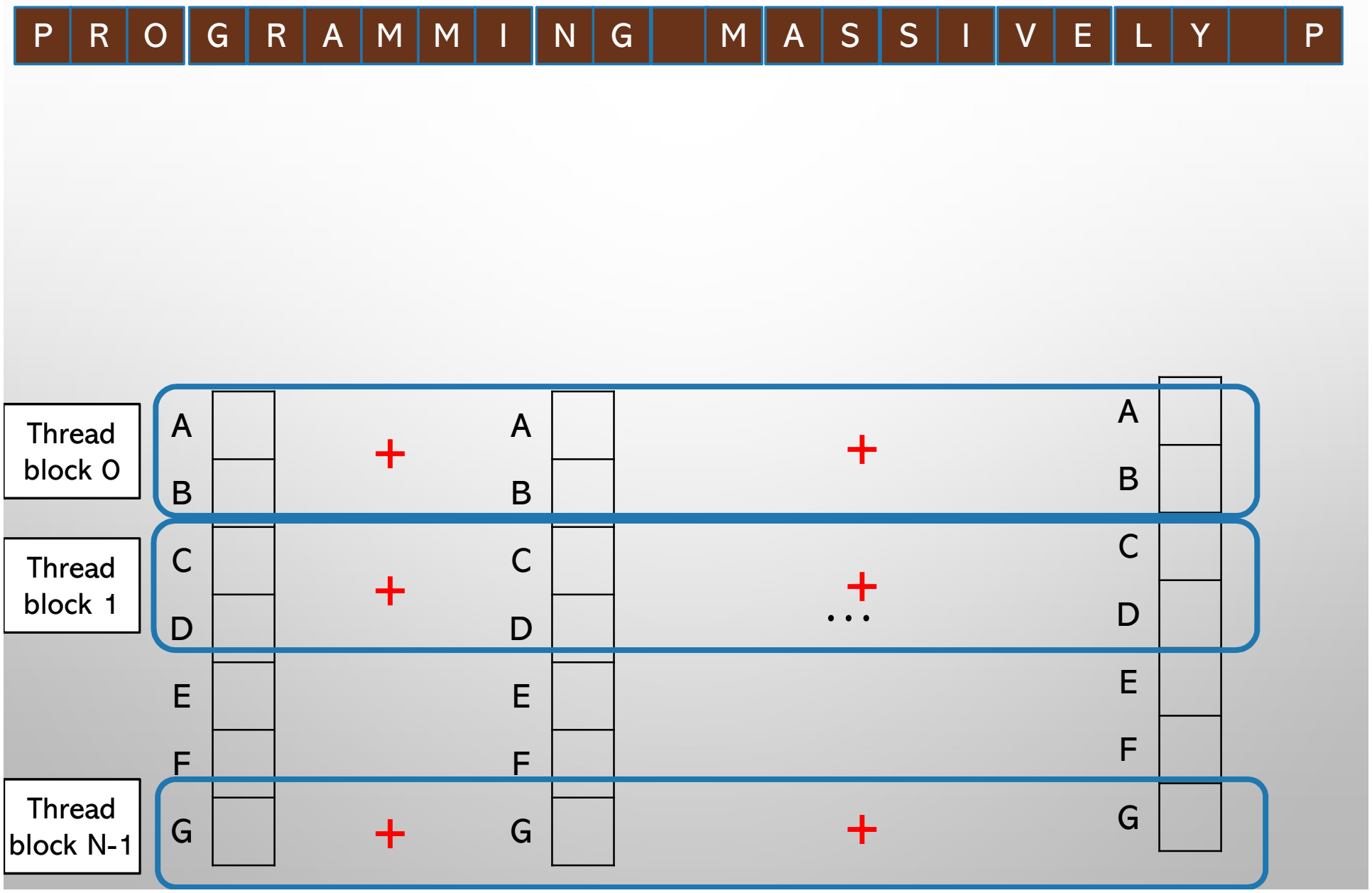
```
__global__ void histogram_v3_global(int *input, int* result, int
     num_blocks) {
  // Calculate global thread ID
  int tid = blockIdx.x * blockDim.x + threadIdx.x;
  // number of threadblock better be one
  for(int bin_id=tid; bin_id < BINS; bin_id += blockDim.x) {
    //we don't use reduce here. parallel reduction might increase
     perf when N is large
    unsigned int total=0;
    for(int i=0;i<num_blocks;i++) {
      total += input[bin_id + i*BINS];
    }
    result[bin_id] = total;
  }
}
```

- Next kernel merges the histograms
- A common technique used in many cases
- I didn't try, but parallel reduction might help here depending on the worksize

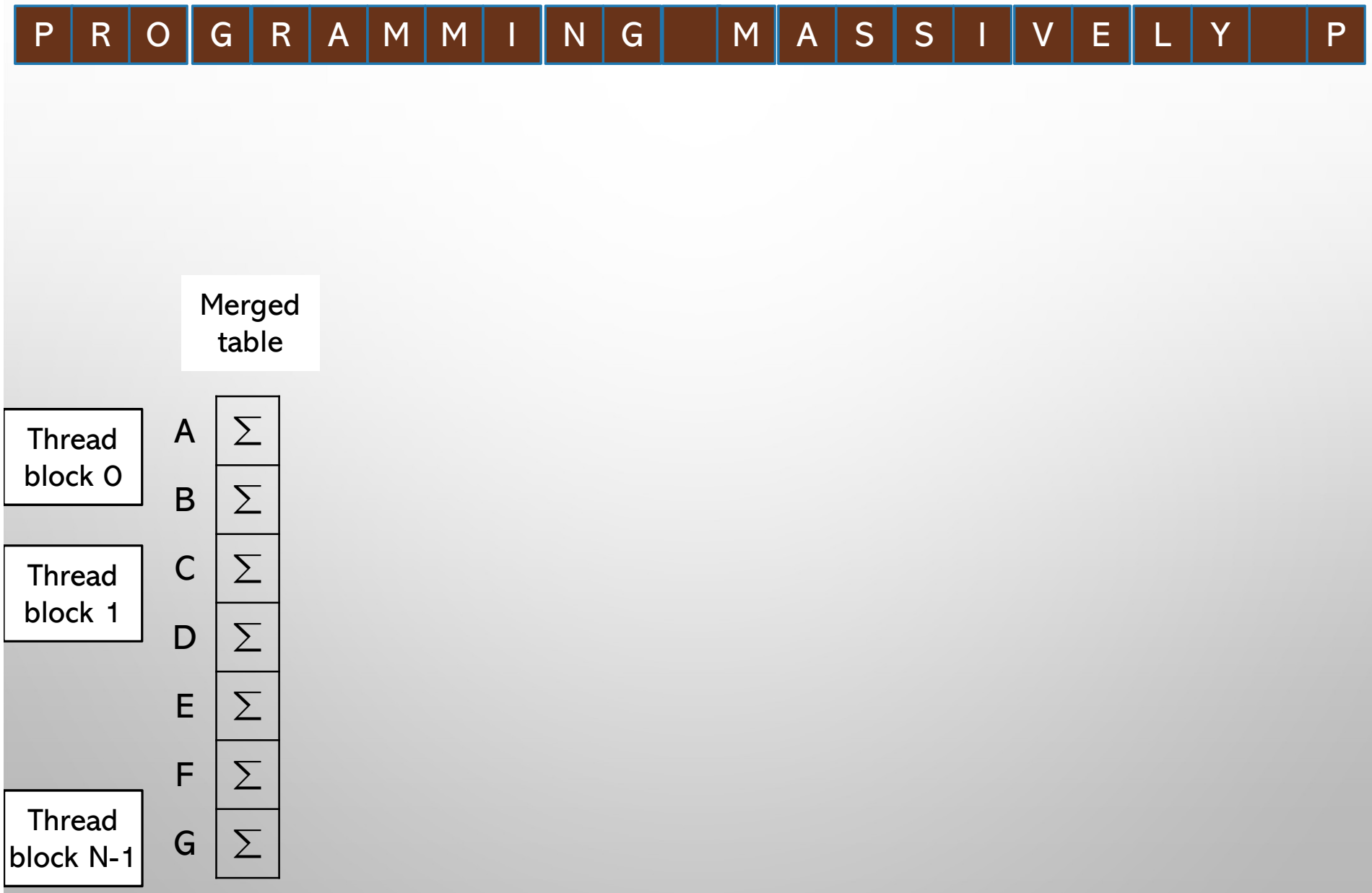
KERNEL SPLIT METHOD



KERNEL SPLIT METHOD



KERNEL SPLIT METHOD



SUMMARY

- Atomics is a useful operation, but has long latency
- Shared memory atomics can improve performance, but needs more engineering